

Business Intelligence SQL Assignment Instructions

First of all, have a look at the SQL database creation and population code that accompanies this

Student Name: Matthew Kolonich

assignment. You should be familiar with the fields and what kinds of data are represented in the database. **All questions and items in this assignment pertain to the SQL code that accompanies this assignment.**

You must include all SQL code here in the spaces below, respecting standard capitalization conventions as presented in the tutorials. **For ease of review, please use BOLD on any SQL code (see the example below). Your SQL code must be in text format, so that the instructor can cut and paste for review and testing purposes. Here is an example of SQL code/text that can be cut and pasted for testing:**

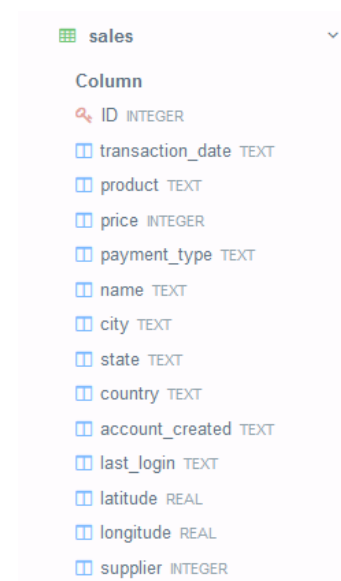
```
CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT, age INTEGER, weight REAL);  
INSERT INTO customers VALUES (73, "Brian", 33, 50);
```

For certain responses you are also asked to include a 'snip' / screenshot of your results. If you can, try not to take an image of the entire screen, but only of the relevant output.

You should use sqliteonline.com to complete this assignment, both to test code and take screenshots:

<https://sqliteonline.com/>

Here is an example of a database schema image and a query results image from sqliteonline.com. Due to screen resolution, it's not always possible to capture all query results. But be sure your images are legible, no smaller than the second image below. If an image is not legible, your assignment may be returned to you and you'll have to rerun the SQL scripts.



```
1 SELECT * FROM sales;
```

#	ID	transacti...	product	price	paymen...	name	city	state	country	account...	last_login	latitude	longitude	supplier
1		1/2/19 4:53	Chair	1200	Visa	Bettina S...	Parkville	MO	United St...	1/2/19 4:42	1/2/19 7:49	39.195	-94.68194	2
2		1/2/19 13:08	Chair	1200	Mastercard	Federica ...	Astoria	OR	United St...	1/1/19 16:21	1/3/19 12:32	46.18806	-123.83	3
3		1/4/19 12:56	Couch	3600	Visa	Gerd Win...	Cahaba H...	AL	United St...	11/15/18 ...	1/4/19 12:45	33.52056	-86.8025	1
4		1/4/19 13:19	Chair	1200	Visa	Laurence ...	Mickleton	NJ	United St...	9/24/18 1...	1/4/19 13:04	39.79	-75.23806	3
5		1/4/19 20:11	Chair	1200	Mastercard	Fleur Bar...	Peoria	IL	United St...	1/3/19 9:38	1/4/19 19:45	40.69361	-89.58889	2
6		1/2/19 20:09	Chair	1200	Mastercard	Adam Ma...	Martin	TN	United St...	1/2/19 17:43	1/4/19 20:01	36.34333	-88.85028	4
7		1/5/19 2:42	Chair	1200	Diners	Stacy Ro...	New York	NY	United St...	1/5/19 2:23	1/5/19 4:59	40.71417	-74.00639	2
8		1/2/19 9:16	Chair	1200	Mastercard	Sean Davis	Shavano ...	TX	United St...	1/2/19 8:32	1/5/19 9:05	29.42389	-98.49333	3
9		1/5/19 10:08	Chair	1200	Visa	Georgia ...	Eagle	ID	United St...	11/11/18 1...	1/5/19 10:05	43.69556	-116.35306	1
10		1/2/19 14:18	Chair	1200	Visa	Richard P...	Riverside	NJ	United St...	12/9/18 1...	1/5/19 11:01	40.03222	-74.95778	4

Feel free to take several screenshots as needed or make the columns narrower in the sqLiteonline output before taking the screenshot (columns can be manually made more narrow by hovering over the lines marked in red below):

```
1 SELECT * FROM sales;
```

#	ID	transacti...	product	price	paymen...	name	city	state	country	account...	last_login	latitude	longitude	supplier
1		1/2/19 4:53	Chair	1200	Visa	Bettina S...	Parkville	MO	United St...	1/2/19 4:42	1/2/19 7:49	39.195	-94.68194	2
2		1/2/19 13:08	Chair	1200	Mastercard	Federica ...	Astoria	OR	United St...	1/1/19 16:21	1/3/19 12:32	46.18806	-123.83	3
3		1/4/19 12:56	Couch	3600	Visa	Gerd Win...	Cahaba...	AL	United St...	11/15/18 ...	1/4/19 12:45	33.52056	-86.8025	1
4		1/4/19 13:19	Chair	1200	Visa	Laurence ...	Mickleton	NJ	United St...	9/24/18 1...	1/4/19 13:04	39.79	-75.23806	3
5		1/4/19 20:11	Chair	1200	Mastercard	Fleur Bar...	Peoria	IL	United St...	1/3/19 9:38	1/4/19 19:45	40.69361	-89.58889	2
6		1/2/19 20:09	Chair	1200	Mastercard	Adam Ma...	Martin	TN	United St...	1/2/19 17:43	1/4/19 20:01	36.34333	-88.85028	4
7		1/5/19 2:42	Chair	1200	Diners	Stacy Ro...	New York	NY	United St...	1/5/19 2:23	1/5/19 4:59	40.71417	-74.00639	2
8		1/2/19 9:16	Chair	1200	Mastercard	Sean Davis	Shavan...	TX	United St...	1/2/19 8:32	1/5/19 9:05	29.42389	-98.49333	3
9		1/5/19 10:08	Chair	1200	Visa	Georgia ...	Eagle	ID	United St...	11/11/18 1...	1/5/19 10:05	43.69556	-116.35306	1

For some of these questions, you will need to perform a SUM or AVERAGE in a group-

<https://www.khanacademy.org/computing/computer-programming/sql/more-advanced-sql-queries/pt/restricting-grouped-results-with-having>

1). 10 points. As you glance through the database, you note that Houston is spelled with a lower case 'h' in at least one case. First of all, write the SQL code to list all values for Houston, capitalized or not capitalized. Begin with **SELECT * FROM sales** to list all fields.

```
SELECT *FROM sales  
WHERE city LIKE '%Houston%'
```

Then, write the SQL script to update all lower case instances of houston to Houston. NOTE: use of **WHERE id=** is not permitted. This method is used in the tutorial but is not practical if you had a very large database.

```
UPDATE sales SET city = REPLACE(city, 'houston', 'Houston')  
WHERE city LIKE '%houston%'
```

2) 10 points. Write an SQL query to list all products and the total sales in dollars for each product. List products in order from highest total sales to lowest. Use Total_Sales as the label for the field (column) containing the sales data.

```
SELECT product, SUM(price) AS Total_Sales FROM sales  
GROUP BY product ORDER BY Total_Sales desc;
```

ists only available with a paid subscription.

SQLite

0.1.4 beta (Mem...

Table

- demo
- sales

Column

- ID INTEGER
- transaction_date TEXT
- product TEXT
- price INTEGER
- payment_type TEXT
- name TEXT
- city TEXT
- state TEXT
- country TEXT
- account_created TEXT

```
1 SELECT product, SUM(price) AS Total_Sales
2 FROM sales
3 GROUP BY product
4 ORDER BY Total_Sales DESC;
```

product	Total_Sales
Bed	64800
Chair	51600
Couch	30200

3) 10 points. Write an SQL query to list the customer name and the dollar amount of total purchases. List only names and dollar amounts if total purchases exceed \$5000. List in order of total purchases from highest to lowest and use the column titles Customer_Name and Total_Purchases.

```
SELECT name as Customer_Name, SUM(price) as Total_Purchases FROM sales
WHERE price > 5000 GROUP BY name
ORDER BY Total_Purchases desc;
```

SQLite 0.14 beta (Mem...)

Table

- demo
- sales

Column

- ID INTEGER
- transaction_date TEXT
- product TEXT
- price INTEGER
- payment_type TEXT
- name TEXT
- city TEXT
- state TEXT
- country TEXT
- account_created TEXT
- last_login TEXT
- latitude REAL
- longitude REAL
- supplier INTEGER
- sqlite_sequence

MariaDB

PostgreSQL

```

1 SELECT name AS Customer_Name, SUM(price) AS Total_Purchases
2 FROM sales
3 WHERE price > 5000
4 GROUP BY name
5 ORDER BY Total_Purchases DESC;

```

Customer_Name	Total_Purchases
Lisa Owens	13200
Regis Ritz	7500
Nikki Sbox	7500
Linda Smith	7500
Dottie Pang	7500
Maxine Klein	7200
Buddy Dyer	7200
Bryan Kerrene	7200

4) 10 points. Write an SQL query to list the states (note Canadian provinces are also included in the database) and the dollar amount of the average purchase. List only states (and provinces) when the average purchase exceeds \$2500. List in order of average purchase from highest to lowest and use the column titles State_Province and Average_Purchase. Round the dollar amount of the Average_Purchase so only whole numbers (no decimals) will appear.

```

SELECT state AS State_Province, ROUND(AVG(price)) AS Average_Purchase FROM sales
WHERE price > 2500 GROUP BY state
ORDER BY Average_Purchase desc;

```

SQLite	1 SELECT state AS State_Province, ROUND(AVG(price)) AS Average_Purchase
0.1.4 beta (Mem...	2 FROM sales
ble	3 WHERE price > 2500
demo	4 GROUP BY state
sales	5 ORDER BY Average_Purchase DESC;
Column	
ID INTEGER	
transaction_date TEXT	
product TEXT	
price INTEGER	
payment_type TEXT	
name TEXT	
city TEXT	
state TEXT	
country TEXT	
account_created TEXT	
last_login TEXT	
latitude REAL	
longitude REAL	
supplier INTEGER	
sqlite_sequence	
ariaDB	
ostgreSQL	
S SQL	

State_Province	Average_Purchase
MD	7500
NY	7350
TX	6067
VT	5550
FL	5550
IL	5400
CA	5400
SK	3600
GA	3600
AL	3600

5a) NOTE: For all of Q5, this must be an actual situation/scenario that can be executed with an SQL query on the data provided in the assignment. NOT a hypothetical scenario or partial SQL query. If this is unclear, please ask.

15 points. In 2-3 sentences, given the nature and structure of the **furniturestore.sql** data, describe a situation or scenario (or question that might be answered) that would require use of a CASE statement

The furniture store has sales in many different cities. They are planning on pushing advertising campaigns in there busier cities. We want to determine which cities have total sales over 5000. This will enable us to see which cities are the busiest.

in SQL. (This is a very open question with many correct responses. Think about what the CASE statement in SQL allows us to do.)

5b) Create the SQL query for the scenario presented in part a). This SQL query must be complete, executable and produce output relevant to the scenario presented in part a).

```
SELECT city, SUM(price) AS Total_Sales
FROM sales
WHERE price > 5000
GROUP BY city
ORDER BY Total_Sales desc;
```

The screenshot shows a database client interface with a sidebar on the left and a main panel on the right. The sidebar lists databases: SQLite (selected), MariaDB, and PostgreSQL. Under SQLite, there are tables: demo, sales (selected), and sqlite_sequence. The main panel displays the SQL query and its results.

```
1 SELECT city, SUM(price) AS Total_Sales
2 FROM sales
3 WHERE price > 5000
4 GROUP BY city
5 ORDER BY Total_Sales DESC;
```

city	Total_Sales
Sugar Land	13200
Woodsboro	7500
Pittsfield	7500
New Rochelle	7500
Miami	7500
New York	7200
Morton	7200
Los Gatos	7200

6a) 15 points. You have been asked to produce a report that shows all chairs bought on January 12, 2019. In the box below, respond to the following question:

Why does the following SQL query not yield any results (feel free to test it)?

SELECT * FROM sales WHERE product = 'Chair' AND transaction_date = '1/12/19';

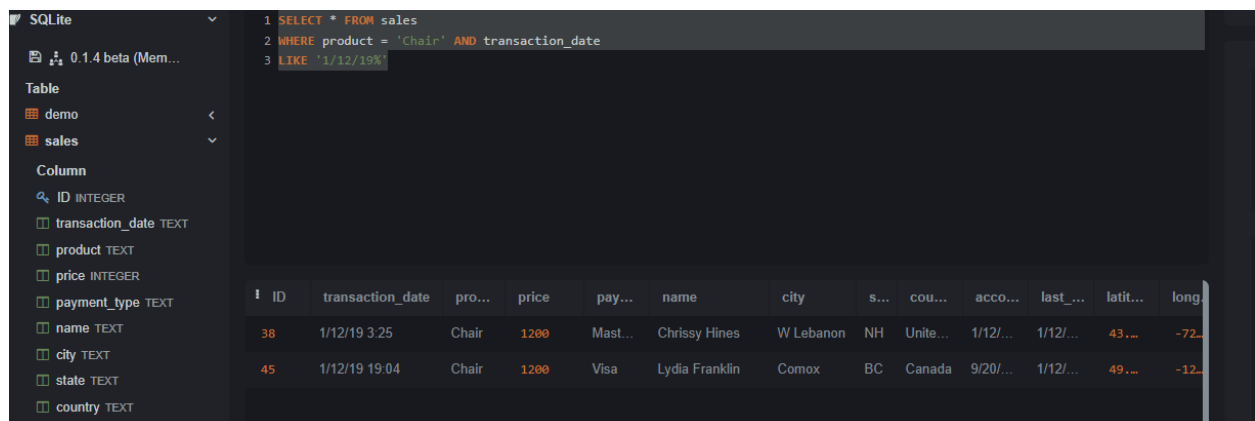
The transaction_date also includes the time in the transaction. The above query is only asked about the date. Since it has to be equal too and the time is not included, they are not equal and there are no results.

6b) Rewrite the query so that it yields the correct results.

SELECT * FROM sales

Where product = 'Chair' AND transaction_date

LIKE '1/12/19%'



The screenshot shows a SQLite database interface. On the left, a sidebar lists the database 'demo' and its tables 'sales'. The 'sales' table is selected, and its columns are listed: ID (INTEGER), transaction_date (TEXT), product (TEXT), price (INTEGER), payment_type (TEXT), name (TEXT), city (TEXT), state (TEXT), country (TEXT), account_number (TEXT), last_name (TEXT), latitude (TEXT), and longitude (TEXT). The main area displays the SQL query: `1 SELECT * FROM sales`, `2 WHERE product = 'Chair' AND transaction_date`, `3 LIKE '1/12/19%'`. Below the query, the results of the query are shown in a table with 13 columns: ID, transaction_date, product, price, payment_type, name, city, state, country, account_number, last_name, latitude, and longitude. Two rows are displayed: one for ID 38 and another for ID 45, both representing chair purchases on 1/12/19.

ID	transaction_date	product	price	payment_type	name	city	state	country	account_number	last_name	latitude	longitude
38	1/12/19 3:25	Chair	1200	Mast...	Chrissy Hines	W Lebanon	NH	Unite...	1/12/...	1/12/...	43....	-72....
45	1/12/19 19:04	Chair	1200	Visa	Lydia Franklin	Comox	BC	Canada	9/20/...	1/12/...	49....	-12....

PART 2:

Part 2 introduces a second table to the database schema. When creating queries from two tables, in particular when there may be a field that shares a common name, it is safer to construct your queries with the table name AND field name. You may wish to review the “JOINing related tables” tutorial and video on Khan Academy.

7) 5 points. Using an SQL script, add (CREATE) a suppliers table to the database schema, with an autoincrement integer as the primary key (id), and fields for name and country (both text). The primary key (id) will correspond to the supplier field in the sales table (e.g. supplier in sales is a foreign key).

```
CREATE table suppliers (id INTEGER PRIMARY key AUTOINCREMENT, name TEXT, country TEXT,
supplier INTEGER, FOREIGN key (supplier) REFERENCES sales(supplier))
```

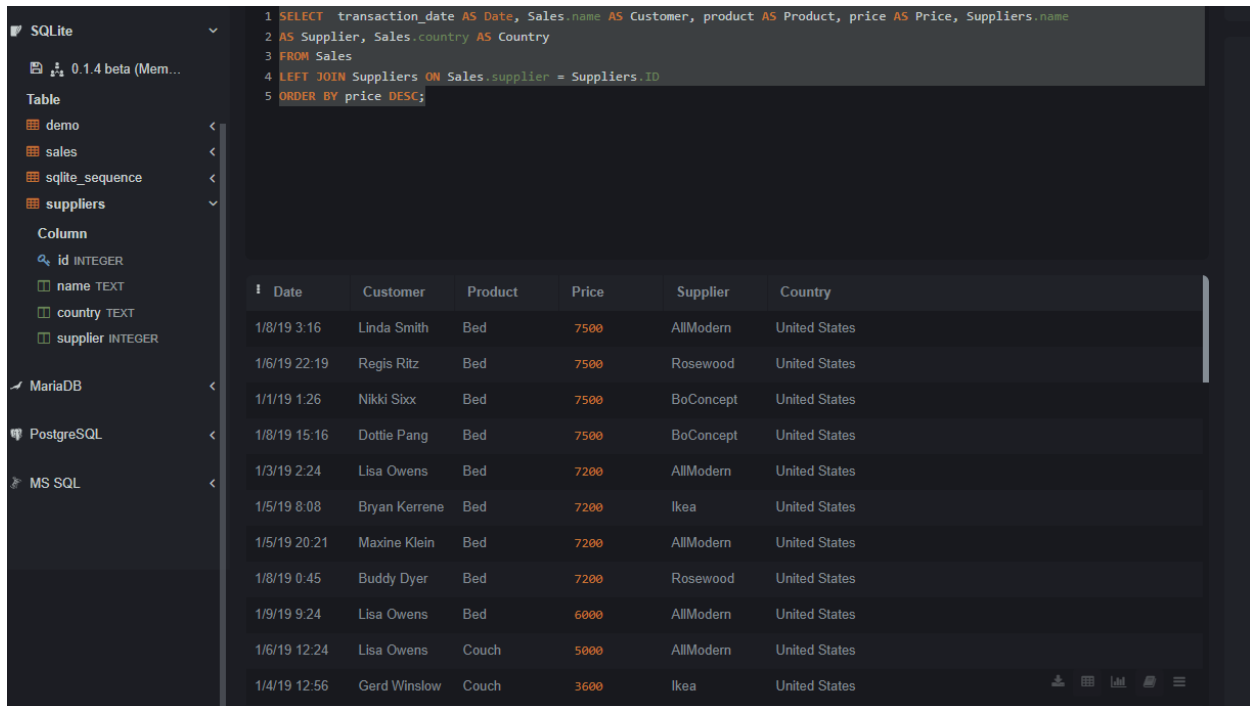
8) 5 points. Using an SQL script, insert the following data into the suppliers table, in the order presented in the table (Reminder: id primary key should have been set to autoincrement when the table was created):

Name	Country
Ikea	Sweden
Rosewood	Thailand
BoConcept	Denmark
AllModern	United States

```
INSERT into suppliers VALUES (1,'Ikea', 'Sweden',1), (2,'Rosewood', 'Thailand',2),
(3,'BoConcept', 'Denmark',3), (4,'AllModern', 'United States',4)
```

9) 10 points. Write an SQL query to list transaction date, customer name, product, price, supplier name and country (6 columns) ordered by price from highest to lowest. List only items with a price greater than 1500. Construct the SQL query to present field (column) titles as: Date, Customer, Product, Price, Supplier, Country.

```
SELECT transaction_date AS Date, Sales.name AS Customer, product AS Product, price AS Price, Suppliers.name  
as Supplier, Sales.country AS Country  
FROM Sales  
LEFT JOIN Suppliers ON Sales.supplier = Suppliers.ID  
ORDER BY price DESC;
```

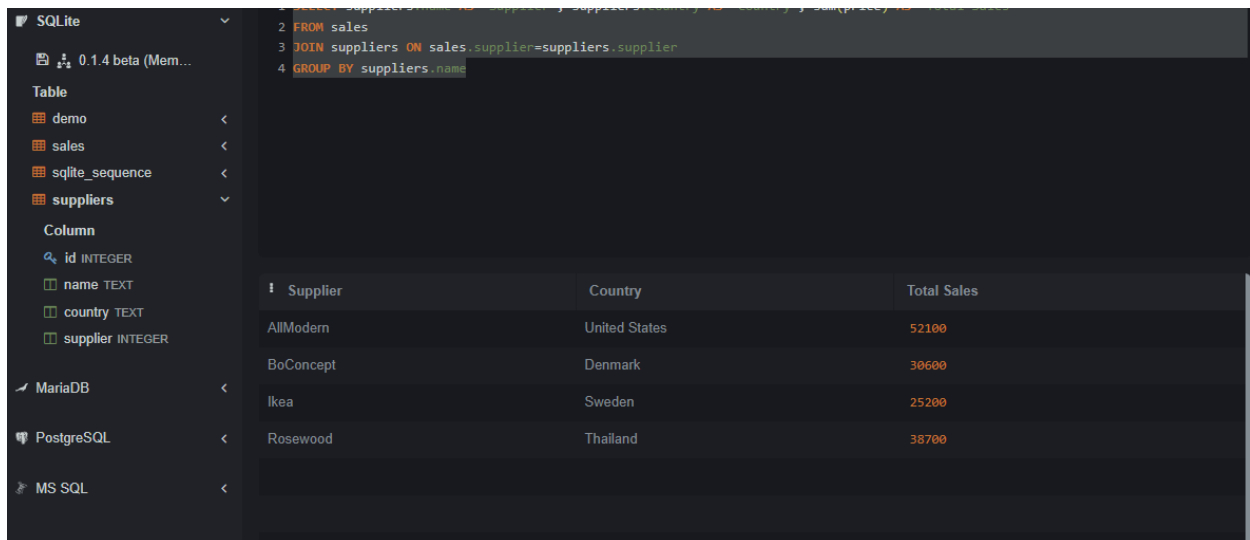


The screenshot shows a database client interface with a sidebar on the left listing databases (SQLite, MariaDB, PostgreSQL, MS SQL) and tables (demo, sales, sqlite_sequence, suppliers). The main area displays the SQL query and its results. The query is:
1 SELECT transaction_date AS Date, Sales.name AS Customer, product AS Product, price AS Price, Suppliers.name
2 AS Supplier, Sales.country AS Country
3 FROM Sales
4 LEFT JOIN Suppliers ON Sales.supplier = Suppliers.ID
5 ORDER BY price DESC;
The results table has 6 columns: Date, Customer, Product, Price, Supplier, and Country. It lists 12 transactions, sorted by price in descending order. The prices range from 7500 down to 3600.

Date	Customer	Product	Price	Supplier	Country
1/8/19 3:16	Linda Smith	Bed	7500	AllModern	United States
1/6/19 22:19	Regis Ritz	Bed	7500	Rosewood	United States
1/1/19 1:26	Nikki Sbcx	Bed	7500	BoConcept	United States
1/8/19 15:16	Dottie Pang	Bed	7500	BoConcept	United States
1/3/19 2:24	Lisa Owens	Bed	7200	AllModern	United States
1/5/19 8:08	Bryan Kerrene	Bed	7200	Ikea	United States
1/5/19 20:21	Maxine Klein	Bed	7200	AllModern	United States
1/8/19 0:45	Buddy Dyer	Bed	7200	Rosewood	United States
1/9/19 9:24	Lisa Owens	Bed	6000	AllModern	United States
1/6/19 12:24	Lisa Owens	Couch	5000	AllModern	United States
1/4/19 12:56	Gerd Winslow	Couch	3600	Ikea	United States

10) 10 points. Write an SQL query to list supplier name, country and total sales for that supplier.

```
SELECT suppliers.name as 'Supplier', suppliers.country as 'Country', sum(price) as 'Total Sales'
from sales
JOIN suppliers on sales.supplier=suppliers.supplier
group by suppliers.name
```



The screenshot shows a database application interface. On the left, a sidebar lists the database 'SQLite' and its tables: 'demo', 'sales', 'sqlite_sequence', and 'suppliers'. The 'suppliers' table is selected, showing its columns: 'id' (INTEGER), 'name' (TEXT), 'country' (TEXT), and 'supplier' (INTEGER). The main area displays the SQL query:
1 SELECT suppliers.name as 'Supplier', suppliers.country as 'Country', sum(price) as 'Total Sales'
2 FROM sales
3 JOIN suppliers ON sales.supplier=suppliers.supplier
4 GROUP BY suppliers.name
The results are shown in a table with three columns: 'Supplier', 'Country', and 'Total Sales'. The data rows are: AllModern (United States, 52100), BoConcept (Denmark, 30600), Ikea (Sweden, 25200), and Rosewood (Thailand, 38700).

Supplier	Country	Total Sales
AllModern	United States	52100
BoConcept	Denmark	30600
Ikea	Sweden	25200
Rosewood	Thailand	38700

Insert (copy and paste) all code from Items 7-10 within this textbox:

```
CREATE table suppliers (id INTEGER PRIMARY key AUTOINCREMENT, name TEXT, country TEXT,  
supplier INTEGER, FOREIGN key (supplier) REFERENCES sales(supplier))
```

```
INSERT into suppliers VALUES (1,'Ikea', 'Sweden',1), (2,'Rosewood', 'Thailand',2),  
(3,'BoConcept','Denmark',3), (4,'AllModern', 'United States',4)
```

```
SELECT transaction_date AS Date, Sales.name AS Customer, product AS Product, price AS Price,  
Suppliers.name  
as Supplier, Sales.country AS Country  
FROM Sales  
LEFT JOIN Suppliers ON Sales.supplier = Suppliers.ID  
ORDER BY price DESC;
```

```
SELECT suppliers.name as 'Supplier', suppliers.country as 'Country', sum(price) as 'Total Sales'  
from sales  
JOIN suppliers on sales.supplier=suppliers.supplier  
group by suppliers.name
```